

## What is PyTorch?

PyTorch is an open-source Python library developed by Meta (Facebook AI Research) for Machine Learning (ML) and Deep Learning (DL). It is used to build, train, and deploy Artificial Intelligence (AI) models.

Developed by: Meta (Facebook AI Research)

Released: 2016

Website: <https://pytorch.org>

## Features of PyTorch

- Open-source and free
- Supports Machine - Learning and Deep Learning
- Dynamic computation graph
- Easy to learn and use
- GPU acceleration using CUDA
- Automatic differentiation (Autograd)
- Strong Python integration
- Supports distributed training
- Works with NumPy and other Python libraries
- Large community support

## Advantages of PyTorch

- Beginner-friendly
- Dynamic computation graph
- Easy debugging
- Fast model development
- Excellent GPU support
- Widely used in research and industry

## Disadvantages of PyTorch

- Slightly slower for production deployment compared to TensorFlow
- Requires more memory for some applications

- Smaller mobile deployment ecosystem than TensorFlow

## Installation

```
pip install torch torchvision torchaudio
```

## Import PyTorch

```
import torch

Check Version

import torch

print(torch.__version__)
```

## Tensor

A Tensor is the basic data structure in PyTorch. It is a multi-dimensional array used to store and process data.

## Example

```
import torch a = torch.tensor([1,2,3]) print(a)
```

## Data Types

PyTorch supports different data types.

```
torch.int32
torch.int64
torch.float32
torch.float64
torch.bool
```

## Example

```
torch.tensor([1,2,3], dtype=torch.float32)
```

## Tensor Operations

## Addition

```
torch.add(a,b)
```

Subtraction

```
torch.sub(a,b)
```

Multiplication

```
torch.mul(a,b)
```

Division

```
torch.div(a,b)
```

Square

```
torch.square(a)
```

Square Root

```
torch.sqrt(a)
```

## Tensor Shape

```
tensor.shape
```

## Reshape Tensor

```
tensor.reshape(2,2)
```

## Convert NumPy Array to Tensor

```
import numpy as np  
  
arr = np.array([1,2,3])  
  
tensor = torch.from_numpy(arr)
```

## Neural Network Module

```
import torch.nn as nn model = nn.Sequential( nn.Linear(10,20), nn.ReLU(), nn.Li
```

## Loss Function

```
loss = nn.MSELoss()
```

Other Loss Functions

CrossEntropyLoss

BCELoss

NLLLoss

## Optimizers

```
import torch.optim as optim
```

```
optimizer = optim.Adam(model.parameters())
```

Other Optimizers

SGD

Adam

RMSprop

Adagrad

## Training Model

```
optimizer.zero_grad() loss.backward() optimizer.step()
```

## Prediction

```
output = model(input)
```

## Save Model

```
torch.save(model.state_dict(), "model.pth")
```

```
model.load_state_dict(torch.load("model.pth"))
```

## Important PyTorch Modules

- torch

- torch.nn
- torch.optim
- torch.utils.data - torch.autograd
- torch.cuda
- torchvision
- torchaudio

## Activation Functions

- ReLU
- Sigmoid
- Softmax
- Tanh
- Leaky ReLU

## Common Layers

- Linear
- Conv2D
- MaxPool2D
- Dropout - Flatten
- LSTM
- Embedding

## GPU Support

### Check GPU

```
torch.cuda.is_available()
```

Move tensor to GPU

```
tensor = tensor.cuda()
```

## ✓ Common PyTorch Functions

| Function                    | Purpose         |
|-----------------------------|-----------------|
| <code>torch.tensor()</code> | Create a tensor |

| Function                     | Purpose                                      |
|------------------------------|----------------------------------------------|
| <code>torch.zeros()</code>   | Create a tensor filled with zeros            |
| <code>torch.ones()</code>    | Create a tensor filled with ones             |
| <code>torch.rand()</code>    | Create a tensor with random values           |
| <code>torch.reshape()</code> | Change the shape of a tensor                 |
| <code>torch.add()</code>     | Perform addition of tensors                  |
| <code>torch.mul()</code>     | Perform multiplication of tensors            |
| <code>torch.sqrt()</code>    | Calculate the square root of tensor elements |
| <code>torch.save()</code>    | Save a trained model to a file               |
| <code>torch.load()</code>    | Load a saved model from a file               |

## Applications of PyTorch

- Deep Learning -Machine Learning
- Artificial Intelligence
- Computer Vision
- Image Recognition
- Object Detection
- Face Recognition
- Natural Language Processing (NLP)
- Speech Recognition
- Robotics
- Healthcare
- Self-driving Cars
- Recommendation Systems

## ✓ PyTorch vs TensorFlow

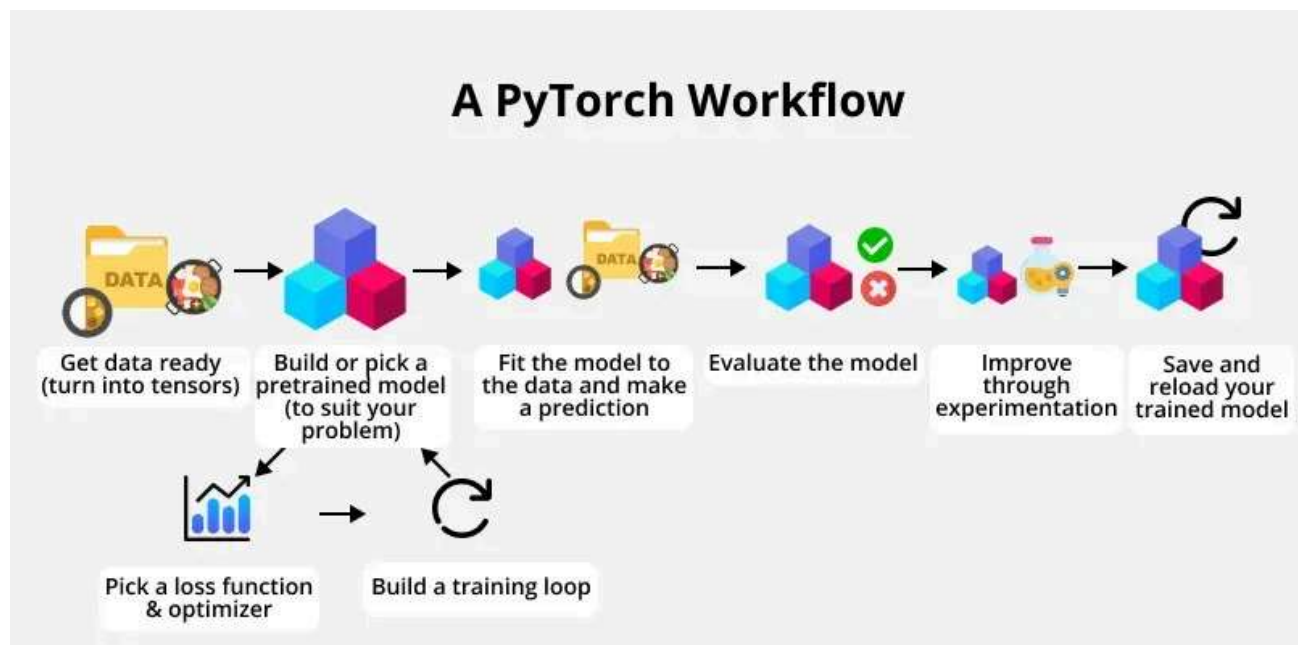
| PyTorch                                         | TensorFlow                                        |
|-------------------------------------------------|---------------------------------------------------|
| Developed by <b>Meta (Facebook AI Research)</b> | Developed by <b>Google</b>                        |
| Uses <b>Dynamic Computation Graph</b>           | Uses <b>Static and Dynamic Computation Graphs</b> |
| Easier to learn and debug                       | Slightly more complex to learn                    |
| Widely used in <b>Research and Academia</b>     | Widely used in <b>Industry and Production</b>     |
| Excellent debugging capabilities                | Better deployment and production tools            |
| Strong Python integration                       | Strong mobile, web, and cloud support             |
| Faster prototyping and experimentation          | Better for large-scale production deployment      |
| Popular for research projects                   | Popular for commercial AI applications            |
| Uses <code>torch.nn</code> for neural networks  | Uses <code>tf.keras</code> for neural networks    |
| Best for flexible model development             | Best for scalable AI solutions                    |

## Future of PyTorch

- Increasing use in Artificial Intelligence and Deep Learning.
- Better GPU support for faster model training.
- More powerful neural network architectures.
- Improved deployment for mobile and cloud platforms.
- Better support for large-scale AI applications. -Growing adoption in healthcare, robotics, and finance.
- Continuous improvements by Meta and the open-source community.
- Increased use in research and academic projects.
- Better integration with AI development tools.
- Growing demand in machine learning and deep learning industries.

## Conclusion

PyTorch is one of the most popular open-source libraries for Machine Learning and Deep Learning. It provides powerful tools for building, training, and deploying AI models. Its dynamic computation graph, easy-to-use interface, and strong GPU support make it a preferred choice for researchers, students, and developers. PyTorch is widely used in computer vision, natural language processing, robotics, healthcare, finance, and artificial intelligence, making it an essential framework for modern deep learning applications.



## Pytorch logo

- List item



- List item